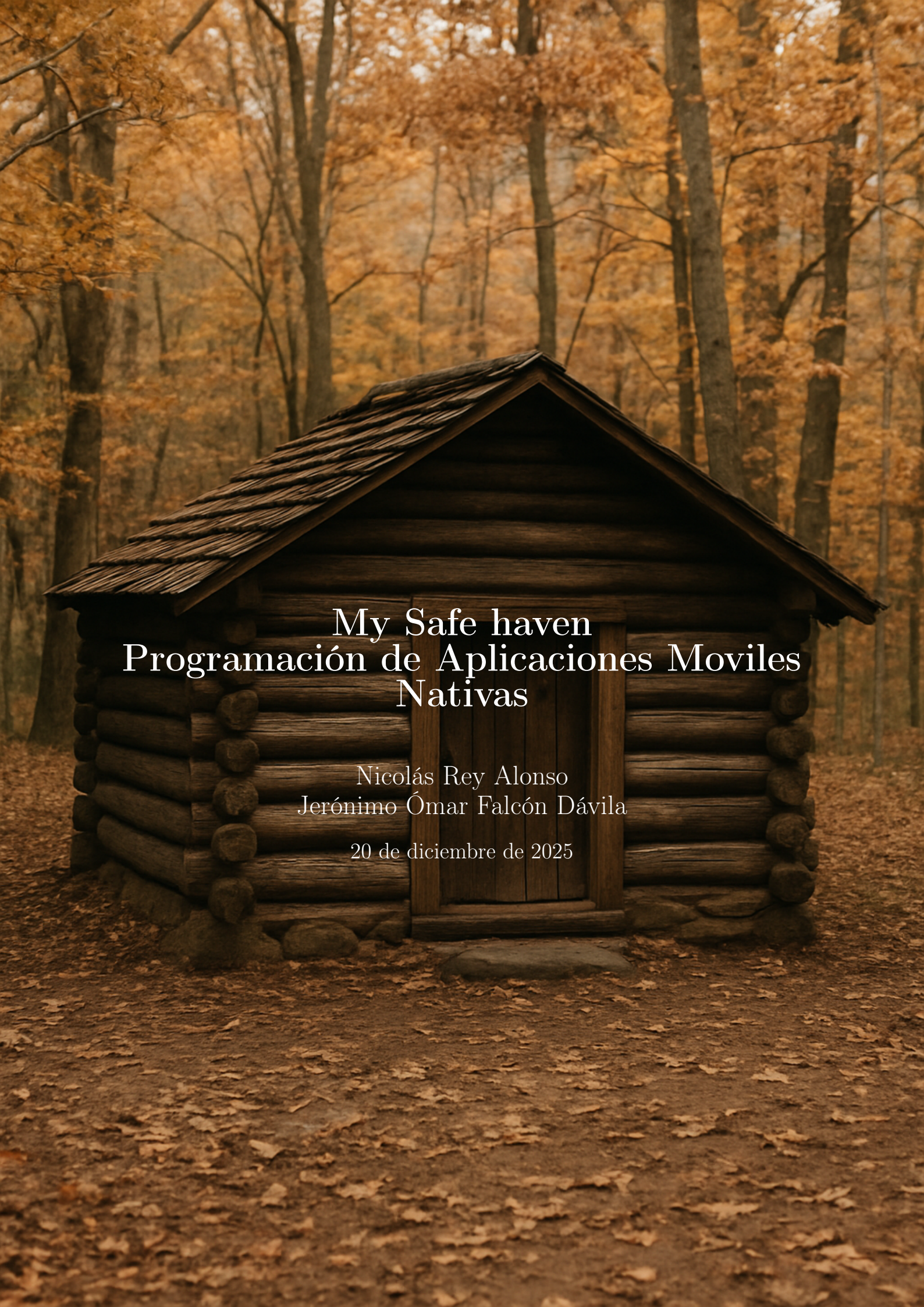


A small, rustic wooden log cabin with a gabled roof made of shingles, situated in a forest. The trees are covered in vibrant autumn foliage in shades of orange, yellow, and brown. The ground is covered with fallen leaves. The cabin has a simple wooden door and is built from horizontal logs.

My Safe haven Programación de Aplicaciones Móviles Nativas

Nicolás Rey Alonso
Jerónimo Ómar Falcón Dávila

20 de diciembre de 2025

Índice

1. Descripción del proyecto	2
2. Plan de Sprint	3
2.1. Plan de trabajo	3
2.2. Sprint 0	4
3. Arquitectura	12
3.1. MVVM	12
3.2. Clean Architecture	12
3.3. Jetpack (Compose/Views)	12
3.4. Estructura del proyecto	13
4. Compilación y despliegue	14
4.1. Pasos para la compilación	14
5. Despliegue	15
5.1. Despliegue local y remoto en máquina física	15
5.2. Despliegue en Amazon Web Services	15
6. Trabajo realizado	19
6.1. Backend	19
6.2. Frontend	19
7. Trabajo pendiente	19
7.1. Backend	19
7.2. Frontend	20

1. Descripción del proyecto

Este proyecto consiste en el desarrollo de una red social integrada en una aplicación móvil nativa denominada *My Safe Haven*. Tomamos como referencia diversas plataformas ampliamente reconocidas, como Instagram, Twitter y Tumblr, adoptando algunos de sus conceptos fundamentales, pero introduciendo un enfoque innovador que diferencia a nuestra aplicación del resto.

La característica central de *My Safe Haven* radica en que el acceso a las publicaciones no es completamente abierto: para visualizar los distintos posts, el usuario debe encontrarse físicamente dentro del “haven” creado por el autor del contenido. Definimos un *haven* como un tipo particular de publicación asociada a un espacio delimitado en el mundo real y compuesta por un nombre identificativo, una descripción, un conjunto de elementos que conforman su propio *feed*, así como coordenadas geográficas y un radio de alcance que determina su área de influencia.

La creación de un haven también requiere presencia física: el usuario debe encontrarse en la ubicación exacta donde desea establecerlo para poder registrarlo en la aplicación. Esta decisión de diseño se inspira en fenómenos virales que combinan elementos digitales con interacción física, como el caso de *Pokémon Go*. Nuestro objetivo es recuperar esa dimensión híbrida entre lo virtual y lo real, incorporándola en un contexto de red social que fomente la exploración, la conexión local y la interacción significativa con el entorno.

De esta manera, *My Safe Haven* busca ofrecer una experiencia social alternativa, basada en la proximidad y en la creación de espacios digitales anclados al mundo físico, promoviendo dinámicas más auténticas y situadas entre usuarios.

2. Plan de Sprint

Para el desarrollo de esta aplicación hemos decidido utilizar la metodología Scrum, ya que nos permite organizar el trabajo de forma iterativa y mantener un seguimiento constante del progreso. Gracias a sus ciclos de trabajo cortos (sprints), podemos adaptarnos con mayor facilidad a los cambios, cumplir con los objetivos definidos para cada etapa y asegurar entregas frecuentes que aporten valor al proyecto.

2.1. Plan de trabajo

Fase	Duración Estimada	Tareas
Plan Sprint Zero	16 Horas	Preparación de documentación, redacción del Plan de Sprint, realización de Mockups y definición de objetivos
Sprint 0	2 Semanas	Desarrollo del mínimo viable de la aplicación

Cuadro 1: Tabla de Plan de Trabajo

2.2. Sprint 0

Las **historias** que se implementan en este sprint son las siguientes:

Loggear en la APP

ID: 0

Prioridad: 0

Estimación: 4h

Historia:

Como: Usuario

Quiero/necesito: Loggearme en la aplicacion

Para: Poder usar la app con mis credenciales

Criterios de validación:

- Cuando no ponga bien la contraseña me debe salir el error.
- Cuando me logge, debe haber alguna indicación de que lo he hecho correctamente.

Base de datos postgres

ID: 1

Prioridad: 0

Estimación: 4h

Historia:

Como: Desarrollador

Quiero/necesito: Una base de datos postgres lanzable en mi pc

Para: Poder desarrollar la aplicación

Criterios de validación:

- Tiene que accederse por el puerto 5243.

Mockups figma

ID: 2

Prioridad: 0

Estimación: 8h

Historia:

Como: Desarrollador

Quiero/necesito: Unos mockups en figma

Para: Poder desarrollar la aplicación

Criterios de validación:

- tienen que representar el diseño de la aplicación.
- tienen que poseer las vista de movil y tablet.
- tiene que contener el tema de colores de la aplicacion.

Obtener mis havens

ID: 3

Prioridad: 0

Estimación: 8h

Historia:

Como: Usuario

Quiero/necesito: obtener mis havens

Para: poder gestionarlos y visualizarlos

Criterios de validación:

- Se deven ver los havens propios y en los que participo.
- Mis havens deven tener un botón que me permitan gestionarlos.
- Los havens deven poderse filtrarion.

Tablero trello

ID: 4

Prioridad: 0

Estimación: 8h

Historia:

Como: Desarrollador

Quiero/necesito: Un tablero trello

Para: realizar el desarrollo de los sprints

Criterios de validación:

- Debe tener todas las historias del sprint 0 a desarrollar.

Plan de sprint

ID: 5

Prioridad: 0

Estimación: 8h

Historia:

Como: Desarrollador

Quiero/necesito: Un plan de sprint

Para: realizar el desarrollo de los sprints

Criterios de validación:

- Debe representar las tareas a realizar.

Backend desplegable en docker

ID: 6

Prioridad: 0

Estimación: 2h

Historia:

Como: Desarrollador

Quiero/necesito: Un backend desplegable en docker

Para: aumentar la portabilidad y la facilidad de despliegue del backend

Criterios de validación:

- Debe representar las tareas a realizar.

Crear Haven

ID: 7

Prioridad: 0

Estimación: 8h

Historia:

Como: Usuario

Quiero/necesito: Crear un haven

Para: Tener mi feed personal

Criterios de validación:

- Si ya hay un haven en el mismo sitio aproximado no debe de poderse crear.
- Al crearse debe mostrarse un mensaje de creado y navegar al feed del haven.
- No debe poderme permitir poner el mismo nombre a dos havens.
- Debo poder poner información de nombre y descripción al haven.
- En la versión gratuita no puedo crear más de dos havens.
- Se me debe crear una feed vacía automáticamente al crear el haven.
- Debe poder ser publico o privado

Eliminar un haven

ID: 8

Prioridad: 0

Estimación: 4h

Historia:

Como: Usuario

Quiero/necesito: Eliminar un haven

Para: Poder gestionar mis havens

Criterios de validación:

- Solo debo poderlo eliminar si es mío.
- Al eliminarlo debe salirme un mensaje de confirmación.
- Se me deben dejar una segundos en los que pueda deshacer la acción.

Editar un haven

ID: 9

Prioridad: 0

Estimación: 4h

Historia:

Como: Usuario

Quiero/necesito: Editar un haven

Para: Cambiar su nombre o descripción

Criterios de validación:

- Solo debo poderlo editar si es mío.
- No debo de poder cambiarle el nombre a uno que ya exista.
- Se me deben dejar unos segundos en los que pueda deshacer la acción.

Añadir posts a mi Haven Feed

ID: 10

Prioridad: 0

Estimación: 4h

Historia:

Como: Usuario

Quiero/necesito: Añadir posts en el feed de mi haven

Para: Que mis amigos lo vean cuando estén en mi haven

Criterios de validación:

- Deben poder ser de tipo texto.
- Deben de poder ser de tipo media (Música, Imagen o Vídeo).
- El texto debe ser enriquecido (Negrita etc).

Visualizar Haven Feeds

ID: 11
Prioridad: 0
Estimación: 8h

Historia:
Como: Usuario
Quiero/necesito: acceder a una haven feed
Para: ver sus publicaciones

Criterios de validación:

- Solo debo de poder verla si estoy físicamente en el haven.
- Si no estoy añadido al haven, no devo de poder verlo.
- Los posts se mostraran de más nuevos a más viejos.

Añadir usuarios a mi haven feed

ID: 12
Prioridad: 0
Estimación: 8h

Historia:
Como: Usuario
Quiero/necesito: poder añadir a usuarios a mi haven feed
Para: que vean mis publicaciones

Criterios de validación:

- Devo de poder enviar invitaciones a amigos.
- Si pasa alguien por mi haven, me debe salir una notificación que me permita invitarle.

Eliminar posts de mi Haven Feed

ID: 13

Prioridad: 0

Estimación: 2h

Historia:

Como: Usuario

Quiero/necesito: poder eliminar posts de mi haven feed

Para: desechar posts que ya no me representan

Criterios de validación:

- Debo de mostrarme un mensaje de confirmación antes de eliminar.
- Debo de tener un tiempo en el que pueda deshacerlo.

Registrarme en My Safe Haven

ID: 14

Prioridad: 0

Estimación: 4h

Historia:

Como: Usuario

Quiero/necesito: registrarme en my safe haven

Para: poder disfrutar de la red social completa

Criterios de validación:

- Debe llegarme un correo de confirmación al mail.
- La contraseña debe de ser segura, y si no lo es mostrar el error.
- El correo electrónico debe de ser válido.

Navegar en la app

ID: 15

Prioridad: 0

Estimación: 4h

Historia:

Como: Usuario

Quiero/necesito: navegar por la aplicación

Para: Poder usar todas sus funciones

Criterios de validación:

- Se deben poder acceder a las páginas principales.
- La navegación deve ser fluida y seguir los principios de M3 Design

3. Arquitectura

Las arquitecturas que hemos decidido utilizar para el desarrollo de esta aplicación son las siguientes:

Clean Architecture + MVVM + Jetpack (Compose/Views)

3.1. MVVM

El patrón *Model–View–ViewModel* (MVVM) nos permite separar de forma efectiva la lógica de presentación de la lógica de la interfaz. Esto facilita el mantenimiento del código y reduce el acoplamiento entre componentes. Las principales ventajas por las que elegimos este patrón son:

- Permite que las vistas contengan únicamente lógica de interfaz, manteniendo un código más limpio y fácil de extender.
- Los *ViewModels* soportan persistencia de estado incluso frente a cambios de configuración, como la rotación de pantalla.
- Incrementa la capacidad de realizar pruebas unitarias al separar las responsabilidades dentro del flujo de datos.

3.2. Clean Architecture

La *Clean Architecture* garantiza una separación clara entre las reglas de negocio, los datos y la interfaz de usuario. Esta división en capas mejora considerablemente la escalabilidad y mantenibilidad del sistema. Lo seleccionamos debido a sus siguientes beneficios:

- Define límites bien estructurados mediante capas independientes: dominio, datos y presentación.
- Permite encapsular la lógica central de negocio en casos de uso (*use cases*), facilitando la evolución del proyecto sin afectar otras capas.
- Facilita la integración de servicios externos, como APIs, sensores de ubicación o bases de datos locales.
- Reduce el acoplamiento, lo que permite sustituir componentes (por ejemplo, cambiar la base de datos local) sin modificar la lógica interna.
- Mejora considerablemente la capacidad de realizar pruebas, especialmente sobre reglas de negocio.

3.3. Jetpack (Compose/Views)

El ecosistema *Jetpack* nos proporciona herramientas modernas y altamente optimizadas para el desarrollo de interfaces y funcionalidades móviles. Dentro del proyecto empleamos tanto *Compose* como vistas tradicionales (*Views*), según las necesidades de cada pantalla. Las razones principales para utilizar estas tecnologías son:

- *Jetpack Compose* permite crear interfaces declarativas, más intuitivas y fáciles de mantener.
- Ofrece integración natural con *ViewModels*, *StateFlow* y otros componentes de arquitectura.
- Simplifica el manejo del estado y el renderizado dinámico de la UI.
- Framework optimizado por Google para el desarrollo moderno de Android, asegurando compatibilidad y evolución a largo plazo.

3.4. Estructura del proyecto

4. Compilación y despliegue

Para poder compilar y ejecutar la aplicación es necesario disponer de los siguientes requisitos:

- Docker CLI o Docker Desktop.
- Android Studio.

De forma adicional, se recomienda contar con los siguientes elementos para facilitar las tareas de depuración y pruebas:

- Python.
- Un dispositivo Android físico o una máquina virtual (emulador).
- Los puertos 5432 y 5050 libres, destinados al backend y al servidor PostgreSQL, respectivamente.

4.1. Pasos para la compilación

Una vez verificados los requisitos previos, el proceso de compilación y despliegue se realiza siguiendo los pasos que se detallan a continuación:

1. Iniciar Docker Desktop o arrancar el *daemon* de Docker.
2. Navegar hasta la carpeta raíz del proyecto y ejecutar el siguiente comando:

```
docker compose up
```

3. Obtener la dirección IP de la máquina en la que se va a ejecutar la aplicación¹.
4. Abrir Android Studio.
5. Ejecutar el proceso de compilación mediante Gradle.
6. Sustituir la IP base configurada en el archivo

```
com.nicojero.mysafehaven.remote.RetrofitClient.kt
```

por la dirección IP obtenida previamente².

7. Ejecutar la aplicación en un dispositivo físico o en un emulador Android.

¹Esta IP será utilizada por la aplicación móvil para comunicarse con el backend.

²Es necesario repetir este paso si la IP de la máquina cambia.

5. Despliegue

El backend desarrollado ha sido probado y desplegado en tres entornos diferentes:

- Entorno local.
- Entorno remoto sobre una máquina física.
- Entorno remoto en la nube mediante Amazon Web Services (AWS).

5.1. Despliegue local y remoto en máquina física

El despliegue en una máquina física remota sigue los mismos pasos que el despliegue en local, descritos previamente en la Sección 4. En ambos casos, la aplicación se ejecuta utilizando contenedores Docker y se comunica con la base de datos mediante los puertos previamente configurados. Lo único de lo que hay que asegurarse es de tener los puertos abiertos en el router y una servicio de dns dinámico, o en su defecto, una ip estática

5.2. Despliegue en Amazon Web Services

Para ejecutar el backend en una plataforma cloud como AWS, es necesario desplegar previamente una infraestructura básica que proporcione los servicios necesarios. Esta infraestructura se define de forma declarativa mediante un template de *AWS CloudFormation*, el cual automatiza la creación de los siguientes recursos:

- Un repositorio **Amazon ECR** para almacenar la imagen Docker del backend.
- Una base de datos **Amazon RDS PostgreSQL**.
- Un clúster **Amazon ECS** con *Fargate* para la ejecución del contenedor.
- La red necesaria (VPC, subredes y grupos de seguridad).

A continuación se muestra un ejemplo de template CloudFormation que implementa esta arquitectura básica:

```
AWSTemplateFormatVersion: "2010-09-09"
Description: Backend con ECR, RDS PostgreSQL y ECS Fargate

Parameters:
  DBUsername:
    Type: String
    Default: appuser

  DBPassword:
    Type: String
    NoEcho: true

  DBName:
    Type: String
    Default: appdb
```

Resources:

VPC:

Type: AWS::EC2::VPC
Properties:
CidrBlock: 10.0.0.0/16

PublicSubnet:

Type: AWS::EC2::Subnet
Properties:
VpcId: !Ref VPC
CidrBlock: 10.0.1.0/24
MapPublicIpOnLaunch: true

BackendSG:

Type: AWS::EC2::SecurityGroup
Properties:
VpcId: !Ref VPC
GroupDescription: Backend access
SecurityGroupIngress:
- IpProtocol: tcp
FromPort: 8080
ToPort: 8080
CidrIp: 0.0.0.0/0

RDSSG:

Type: AWS::EC2::SecurityGroup
Properties:
VpcId: !Ref VPC
GroupDescription: RDS access
SecurityGroupIngress:
- IpProtocol: tcp
FromPort: 5432
ToPort: 5432
SourceSecurityGroupId: !Ref BackendSG

DBSubnetGroup:

Type: AWS::RDS::DBSubnetGroup
Properties:
DBSubnetGroupDescription: RDS subnet group
SubnetIds:
- !Ref PublicSubnet

PostgreSQL:

Type: AWS::RDS::DBInstance
Properties:
Engine: postgres
DBInstanceClass: db.t3.micro
AllocatedStorage: 20
DBName: !Ref DBName
MasterUsername: !Ref DBUsername

```

    MasterUserPassword: !Ref DBPassword
    VPCSecurityGroups:
      - !Ref RDSSG
    DBSubnetGroupName: !Ref DBSubnetGroup
    PubliclyAccessible: true
    BackupRetentionPeriod: 0

ECRRepository:
  Type: AWS::ECR::Repository
  Properties:
    RepositoryName: backend-repo

ECSCluster:
  Type: AWS::ECS::Cluster

TaskExecutionRole:
  Type: AWS::IAM::Role
  Properties:
    AssumeRolePolicyDocument:
      Statement:
        - Effect: Allow
          Principal:
            Service: ecs-tasks.amazonaws.com
          Action: sts:AssumeRole
    ManagedPolicyArns:
      - arn:aws:iam::aws:policy/service-role/AmazonECSTaskExecutionRolePolicy

TaskDefinition:
  Type: AWS::ECS::TaskDefinition
  Properties:
    RequiresCompatibilities:
      - FARGATE
    Cpu: "256"
    Memory: "512"
    NetworkMode: awsvpc
    ExecutionRoleArn: !GetAtt TaskExecutionRole.Arn
    ContainerDefinitions:
      - Name: backend
        Image: !Sub "${AWS::AccountId}.dkr.ecr.${AWS::Region}.amazonaws.com/backend-repo:latest"
        PortMappings:
          - ContainerPort: 8080
        Environment:
          - Name: DATABASE_URL
            Value: !Sub
              - "postgres://${User}:${Pass}@${Host}:5432/${DB}"
              - User: !Ref DBUsername
                Pass: !Ref DBPassword
                Host: !GetAtt PostgreSQL.Endpoint.Address
                DB: !Ref DBName

```

```
ECSService:
  Type: AWS::ECS::Service
  DependsOn: PostgreSQL
  Properties:
    Cluster: !Ref ECSCluster
    LaunchType: FARGATE
    DesiredCount: 1
    TaskDefinition: !Ref TaskDefinition
    NetworkConfiguration:
      AwsvpcConfiguration:
        AssignPublicIp: ENABLED
      Subnets:
        - !Ref PublicSubnet
      SecurityGroups:
        - !Ref BackendSG

Outputs:
  ECRUri:
    Value: !GetAtt ECRRepository.RepositoryUri

  RDSEndpoint:
    Value: !GetAtt PostgreSQL.Endpoint.Address
```

Listing 1: Template CloudFormation de ejemplo para el despliegue del backend en AWS

6. Trabajo realizado

Durante el desarrollo de la aplicación se priorizó la disponibilidad de una demostración funcional o *mínimo producto viable* (MVP) para la presentación final. Este enfoque permitió mostrar las funcionalidades principales de la aplicación, teniendo en cuenta que se optó por implementar una idea original y que el tiempo de desarrollo disponible fue limitado en relación con el *backlog* inicialmente planteado.

6.1. Backend

En el backend se ha logrado implementar la mayoría de las funcionalidades necesarias para el correcto funcionamiento de la aplicación, permitiendo la gestión de usuarios, la comunicación con la base de datos y la exposición de los servicios requeridos por el frontend.

6.2. Frontend

En el frontend también se han implementado la mayor parte de las funcionalidades previstas inicialmente. Entre las principales características desarrolladas se encuentran:

- Operaciones CRUD de *havens*.
- Visualización del estado de la cuenta del usuario y de la disponibilidad para la creación de nuevos *havens*.
- Sistema de mensajería (chat).
- Subida y descarga de imágenes.
- Funcionalidades de geolocalización.

7. Trabajo pendiente

A pesar de los avances realizados, existen diversas funcionalidades que no han podido ser implementadas debido a las limitaciones de tiempo y alcance del proyecto.

7.1. Backend

En el backend quedan pendientes las siguientes mejoras y ampliaciones:

- Implementación de un sistema de autenticación de doble factor mediante correo electrónico u otros mecanismos.
- Sistema de actualización de la cuenta a modo *premium* basado en el resultado de una pasarela de pago, en lugar de una actualización directa en la base de datos.
- Integración completa de una pasarela de pago.
- Mejoras adicionales de seguridad.
- Implementación de WebSockets para la generación de notificaciones en tiempo real.

7.2. Frontend

En el frontend han quedado pendientes diversos aspectos relacionados con la experiencia de usuario (*user experience*):

- Actualización del sistema de chat en tiempo real.
- Uso de WebSockets para la recepción de notificaciones.
- Implementación de un radar de *havens*. Inicialmente, los *havens* cercanos no estaban pensados para mostrarse en una lista, sino mediante una visualización tipo radar. Debido a las restricciones de tiempo, esta funcionalidad no pudo ser desarrollada.